Fine-Tuning LLMs

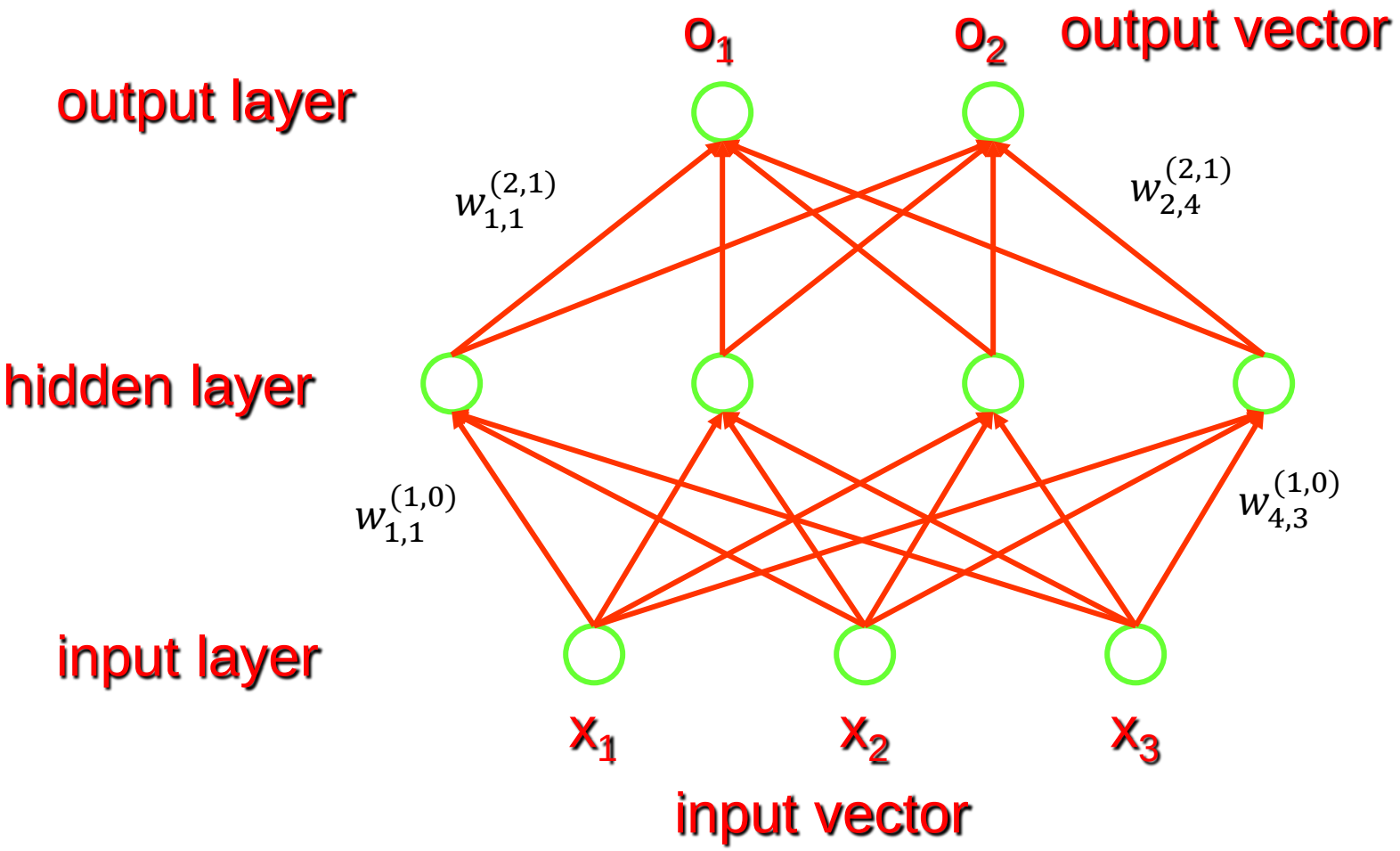
Full Fine-Tuning, LoRA and QLoRA

Prof. Surya Prakash

IIT Indore

Backpropagation Learning

- **Example:** Network function f: $\mathbf{R}^3 \rightarrow \mathbf{R}^2$



Introduction

- Training of Neural networks (learning of weights)
  - Use of Backpropagation

- Backpropagation
  - It is the core algorithm used to train neural networks.
  - It is the algorithm using which models like deep learning learn from errors and improve over time.
  - **It computes gradients of the loss function (error) with respect to each weight** in a neural network.
  - These gradients are then used to update weights using optimization methods like gradient descent.

Backpropagation Learning

- For input pattern p, the i-th input layer node holds $x_{p,i}$.

Net input to j-th node in hidden layer:

$$net_j^{(1)} = \sum_{i=0}^{n} w_{j,i}^{(1,0)} x_{p,i}$$

Output of j-th node in hidden layer:

$$x_{p,j}^{(1)} = S\left(\sum_{i=0}^{n} w_{j,i}^{(1,0)} x_{p,i}\right)$$

Net input to k-th node in output layer:

$$net_k^{(2)} = \sum_{j} w_{k,j}^{(2,1)} x_{p,j}^{(1)}$$

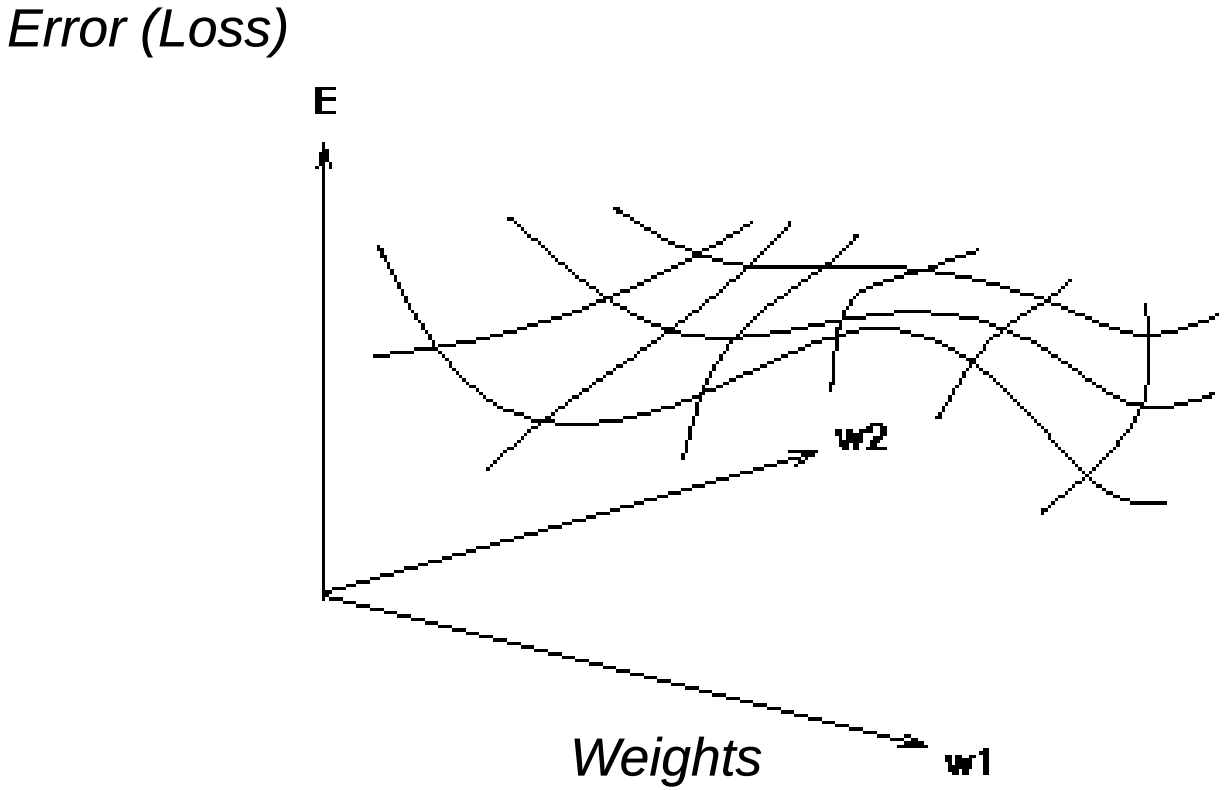
Output of k-th node in output layer:

$$o_{p,k} = S\left(\sum_{j} w_{k,j}^{(2,1)} x_{p,j}^{(1)}\right)$$

Network error (loss) for p:

$$L_p = \sum_{k=1}^{K} (l_{p,k})^2 = \sum_{k=1}^{K} (d_{p,k} - o_{p,k})^2$$

Error (Loss) Surface



Error as function of weights in multidimensional space

Backpropagation Learning

- As E is a function of the network weights, we can use gradient descent to find those weights that result in minimal error.

- For individual weights in the hidden and output layers, we should move against the error gradient (*omitting index p to simplify the expression*):

$$\Delta w_{k,j}^{(2,1)} \propto \frac{-\partial L}{\partial w_{k,j}^{(2,1)}}$$

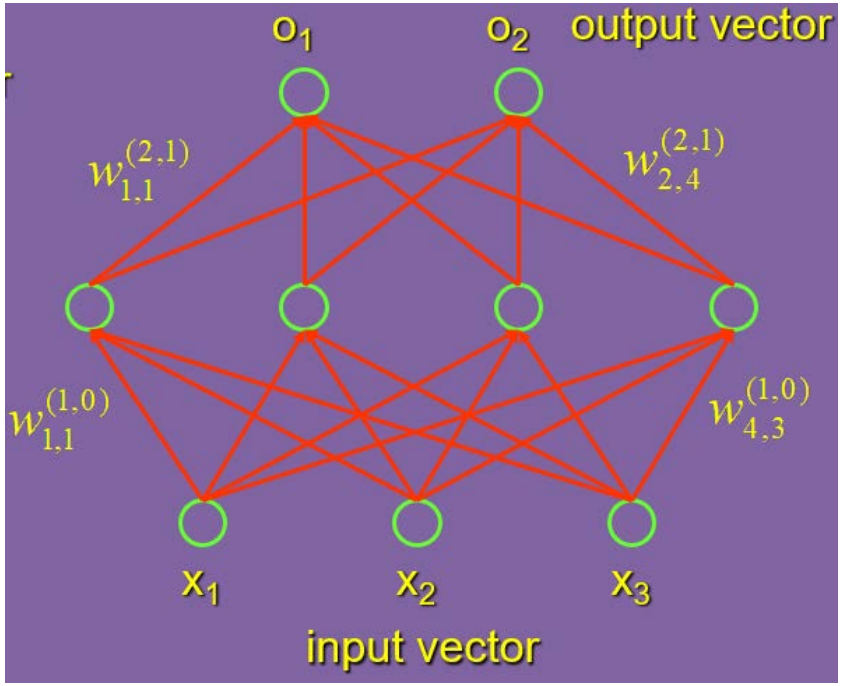
Output layer:
Derivative easy to calculate

$$\Delta w_{j,i}^{(1,0)} \propto \frac{-\partial L}{\partial w_{j,i}^{(1,0)}}$$

Hidden layer:
Derivative difficult to calculate

Backpropagation Learning

- Final weight update form



$$w_{k,j}^{(2,1)} \leftarrow w_{k,j}^{(2,1)} + \Delta w_{k,j}^{(2,1)}$$

where: $\Delta w_{k,j}^{(2,1)} = -\eta \frac{\partial L}{\partial w_{k,j}^{(2,1)}}$

$$w_{j,i}^{(1,0)} \leftarrow w_{j,i}^{(1,0)} + \Delta w_{j,i}^{(1,0)}$$

where $\Delta w_{j,i}^{(1,0)} = -\eta \frac{\partial L}{\partial w_{j,i}^{(1,0)}}$

1. Fine-Tuning LLMs

- Large language models (LLMs) like GPT, LLaMA, or Falcon are **pretrained** on huge corpora.
- Pretrained models are general-purpose
- Fine-tuning adapts them to **specific tasks**:
  - Sentiment analysis
  - Question answering (FAQ bots)
  - Summarization
  - Domain-specific knowledge (medical, legal)

Approaches to Fine-Tuning LLM

- Full Fine-Tuning
  - Update all model parameters
- LoRA (Low-Rank Adaptation)
  - Add small trainable matrices to existing weights
- QLoRA (Quantized LoRA)
  - LoRA + quantization (4-bit/8-bit)
- Prompt Tuning
  - **Add learnable soft prompt vectors at the input only** and keep the model fixed
- Prefix Tuning
  - **add learnable prefix vectors inside every transformer layer (attention layers)** to guide the model

Full Fine-Tuning

- We load the pretrained model
  - Pretrained model: GPT, LLaMA, Falcon, etc
- Train it further on our task/data
- Update every parameter (100%)

Full Fine-Tuning - Mathematical View

- If original model has parameters:

$$\boldsymbol{\theta}$$

- During fine-tuning:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$$

  - $L(\boldsymbol{\theta})$:loss on your dataset
  - ∇_{θ} :gradient w.r.t. *all parameters*
  - η: learning rate

- Unlike LoRA/QLoRA, **no freezing**
  - everything changes here.

Full Fine-Tuning - Issues

- **Catastrophic forgetting** of pre-trained knowledge
  - **Catastrophic:** sudden and severe damage or failure.
- High compute & memory requirement
- Slow training and expensive
- Overfitting on small data

Low-Rank Adaptation (LoRA)

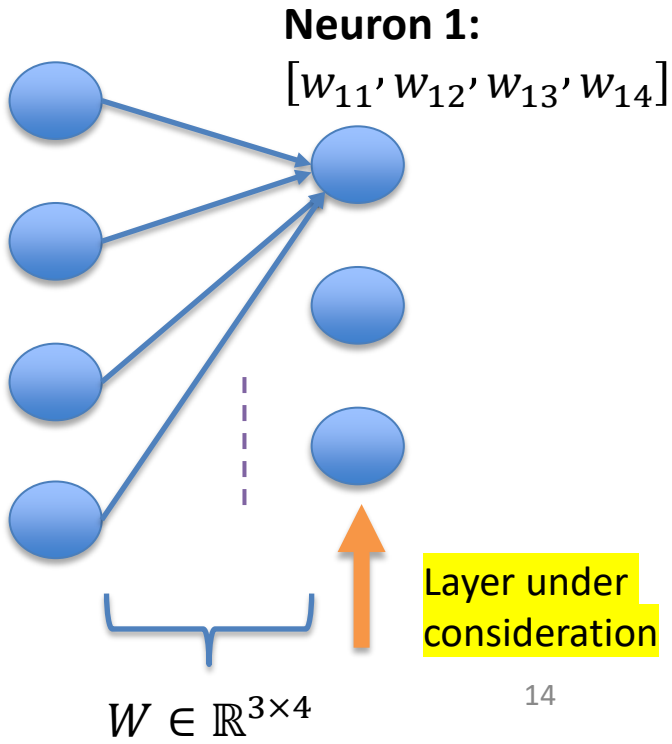
- Instead of fine-tuning the entire large model, **LoRA freezes the original weights** and learns **small low-rank matrices** to adapt the model.

Low-Rank Adaptation (LoRA)

- In a neural network (especially transformers), we have weight matrices: $W \in \mathbb{R}^{d \times k}$
  - Here, W is a single weight matrix with respect to one layer
  - d → number of output features (rows)
  - k → number of input features (columns)

- **What "one layer" really means here?**
  - "one layer" refers to a **single weight matrix inside a neural network layer**, not the entire layer as a block.
  - Suppose a layer has: Input vector size = **4,**
  - Output vector size = **3** (weight matrix: $W \in \mathbb{R}^{3 \times 4}$)

- **Inside that ONE layer**
  - Each output neuron has its own weights:
  - **Neuron 1:** $[w_{11}, w_{12}, w_{13}, w_{14}]$
  - **Neuron 2:** $[w_{21}, w_{22}, w_{23}, w_{24}]$
  - **Neuron 3:** $[w_{31}, w_{32}, w_{33}, w_{34}]$
  - Stack them → gives the full matrix W



LoRA - How it works

- LoRA replaces weight update with:

$$W' = W + \Delta W$$

- Instead of learning full ΔW, LoRA approximates it as:

$$\Delta W = A \times B$$

where:

– $A \in \mathbb{R}^{d \times r}$

– $B \in \mathbb{R}^{r \times k}$

– $r \ll d, k$ ← **low rank**

LoRA - How does it works?

- In real models:
  - $k = d = 4096$
  - Full params = ~16 million
  - LoRA (r=8) ≈ **~65K params only**
  - Huge savings

- How is rank r chosen in LoRA?
  - r is a **hyperparameter** (you choose it, not learned automatically)

- Important points:
  - r controls how much the model can adapt
  - It defines the **capacity of the update** $\Delta W = AB$
  - Think of it as:
    - Small r→ limited learning
    - Large r→ more expressive (closer to full fine-tuning)

What changes in gradient descent with LoRA?

- **Normal training:**

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

- You update the **full weight matrix** W

- **In LoRA,** We write:

$$W' = W + AB$$

  - $W \rightarrow$ frozen (no update), $A, B \rightarrow$ trainable

- **Forward pass**
  - Instead of:

$$y = Wx$$

  - We compute:

$$y = (W + AB)x = Wx + ABx$$

What changes in gradient descent with LoRA?

- Backward pass (Gradient Descent)
  - We compute loss: $L(y)$
- Step 1: Gradient w.r.t output $\frac{\partial L}{\partial y}$ (starting point of backpropagation)
- Step 2: Gradient w.r.t A and B
- Using chain rule:
  - **For A:** $\frac{\partial L}{\partial A} = \frac{\partial L}{\partial y} \cdot (Bx)^T$
  - **For B:** $\frac{\partial L}{\partial B} = A^T \cdot \frac{\partial L}{\partial y} \cdot x^T$
- **Important:**
  - $\frac{\partial L}{\partial W}$ *is ignored* because: W is frozen, No update applied to W

LoRA - Summary

- LoRA:
  - Freeze pretrained model weights
  - Inject **low-rank matrices** for task adaptation
  - Reduces trainable parameters → cheaper

Quantized Low-Rank Adaptation (QLoRA)

- LoRA:
  - Keep full model, learns small updates
- QLoRA:
  - Compress model + learn small updates
  - Same as LoRA + **quantizes model weights** (4-bit / 8-bit)
  - Enables **fine-tuning of very large LLMs** on smaller GPUs

How Does QLoRA works?

- Step 1: Quantize the base model
  - Original weights $W \rightarrow$ converted to **4-bit precision**
  - These weights are **frozen**

- Step 2: Add LoRA adapters
  - $W' = W_{quantized} + AB$
  - $A, B \rightarrow$trainable
  - $W_{quantized} \rightarrow$fixed

- Step 3: Training
  - Forward pass uses quantized weights
  - Gradients update w.r.t. **only A and B**

QLoRA Key Features

- Massive memory savings
  - 16-bit → 4-bit ⇒ **~4× reduction**
  - Enables training: 7B, 13B, even 65B models on limited GPUs

- Near-LoRA performance
  - Accuracy ≈ **LoRA / full fine-tuning**
  - Very small performance drop

How does quantization help?

- In neural networks (including LLMs), weights are **NOT large integers like 100, or 1012**

- They are typically:
  - Small **floating-point numbers**
  - Usually in range like: [-5,5] or even [-1, +1]

- **Why FP16 works?**
  - **Small value range**
  - Weights are usually in: [-5,5] or even [-1, +1]
  - FP16 can **represent this range easily**
    - **FP16 (16-bit Floating Point)** - a numerical format used to store real numbers using **16 bits**

FP16 (16-bit floating point) structure:
•1 bit → sign
•5 bits → exponent
•**10 bits → fraction (mantissa)**
➔ These **10 bits** allow:
•Fine-grained representation of decimals

How does quantization help?

- **Quantization reduces the number of bits used to store weights**, which directly saves memory and computation.

- Normally:
  - Each weight = **16-bit (or 32-bit) float**
- With quantization:
  - Each weight = **4-bit (in QLoRA)**

- Memory reduction
- Example:
  - 16-bit → 4-bit, Memory becomes: $\frac{4}{16} = \frac{1}{4}$
  - 75% memory saving

How does quantization help?

- What actually happens?
  - Instead of storing exact floating-point values, we store a compressed representation (quantized code and scale), and during computation, dequantization reconstructs an approximate value.
  - So, instead of storing exact values like: 0.123456 (float), we store an **approximate value** using fewer bits: 0.12 (quantized), and dequantization reconstructs an approximate value when required for computation.

- Slight loss in precision, but:
  - Huge gain in efficiency

How does quantization help?

- Fit large models in small GPU
  - Example: 7B model:
    - Full precision (16-bit) → ~14 GB
    - With 4-bit precision → ~5 GB

- Quantization approximates the whole value to the nearest representable level, affecting both integer and decimal parts, not just the decimal portion.

How Reduction Happens - Step-by-Step Example

- **1. Take a group of weights**
  - Instead of all weights together, take a **small block**:
  - $W = [0.2, -0.5, 1.0, 0.3]$
- **2. Find range**
  - $\min = -0.5, max = 1.0$
- **3. Compress into 16 levels (4-bit)**
  - 4-bit → only **16 possible values (0–15)**
  - We map real numbers → these 16 levels

How Reduction Happens - Step-by-Step Example

- **4. Store scale + quantized values**
  - Instead of storing: original FP16 values
  - We store:
    - **scale (1 number)**
    - **4-bit integers (0–15)**

- **5. During computation (dequantization) ←**
  - Recover approximate value: $W' = (quantized\ value) \times scale + offset$
  - So: Storage = 4-bit
  - Computation ≈ higher precision

How Reduction Happens - Step-by-Step Example

- **Step 1: Original (High Precision – 16-bit)**
  - Assume some weights (stored in FP16):
  - $W = [0.12, -1.35, 2.78, -0.67]$

- **Step 2: Find Range**
  - min $= -1.35, max = 2.78$
  - Range: range $= 2.78 - (-1.35) = 4.13$

How Reduction Happens - Step-by-Step Example

- **Step 3: 4-bit Representation**
  - 4-bit → $2^4 = 16$ levels
  - So we map values into integers: $0,1,2,\ldots,15$

- **Step 4: Compute Scale**
  - scale $= \frac{range}{15} = \frac{4.13}{15} \approx 0.275$

How Reduction Happens - Step-by-Step Example

- **Step 5: Quantize Values**

$$\text{Formula: } q = round\left(\frac{w - min}{scale}\right)$$

- **Compute:**

Original	Calculation	Quantized (q)
0.12	(0.12 +1.35)/0.275 ≈ 5.35	5
-1.35	0	0
2.78	15	15
-0.67	(0.68)/0.275 ≈ 2.47	2

- **So,**
  - Original: W=$[0.12, -1.35, 2.78, -0.67]$
  - Quantized values: W= [5,0,15,2]

How Reduction Happens - Step-by-Step Example

- Step 6: Dequantization (Recover Approx Values)

$$W' = q \times scale + min$$

- Compute:

q	Reconstructed Value
5	(5×0.275 -1.35 ≈ 0.025)
0	(-1.35)
15	(2.78)
2	(2×0.275 -1.35 ≈ -0.80)

- Final Comparison

Original	Reconstructed
0.12	0.025
-1.35	-1.35
2.78	2.78
-0.67	-0.80

Observation

- Some values are **exact** (min/max)
- Others have **small error**
- Error is **acceptable in deep models**

- So, Even though: Precision dropped from **16-bit → 4-bit**
  - Structure is preserved
  - Errors are small
  - Model still works well

Comparison LoRA and QLoRA

Feature	LoRA	QLoRA
Base model precision	16-bit	4-bit
Memory usage	High	Very low
Accuracy	High	Almost same
Training speed	Faster	Slightly slower

Thank You